MUSANAD CONTRACTS

Risk Signal Workflow

How a real-world event becomes an actionable advisory

Companion notes for the CLM + ECIP Technical Architecture deck. Part 1 walks the risk-signal flow as a story (slide 4). Part 2 unpacks the three concepts on the Logical Architecture slide (slide 2) that don't explain themselves: cross-cutting concerns, async backbone, and why this isn't a microservices architecture.

The story: Monday, 07:42 AM

Layla (Legal Counsel) hasn't even opened her laptop yet, but two things just happened in the platform's background.

Outside the building - UAE's National Center of Meteorology (NCM) publishes a severe-weather advisory for the Hormuz / Fujairah corridor. A shamal sandstorm with sustained winds >40 knots, expected to last 72 hours.

Inside the building - Mubadala's SAP S/4 finance module emits an event: invoice #INV-2026-0418 on contract OQOOD-2026-003 is now 27 days past due.

Two completely different sources. The platform treats them identically.

Stage 1 - INGEST "Catch the signal"

Two listeners woke up at the same time.

- **source-fetch worker** - noticed it was time for its NCM cycle, called the OSINT adapter, parsed the XML, and dropped a row into osint_signal with signal_kind = 'weather.severe_warning' and a JSON blob containing the affected area, wind speed and duration.
- **SAP integration** - (registered in the Platform Admin 'Internal Systems' page as sap_s4_finance, kind = erp) pushed its event through fn_internal_signal_resolve. Even though it came from inside, it lands in the same osint_signal table - just with signal_kind = 'payment_delay' and a different metadata shape.

Either way: a row hits a single table, and the database fires off a PG NOTIFY 'osint_signal_inserted' - basically a 'hey, anyone listening?' broadcast.

The point: *the platform doesn't care whether a risk signal came from CNN, Reuters, OFAC, or the customer's own SAP. They all become rows in the same inbox.*

Stage 2 - CORRELATE "So what?"

The correlation-evaluator worker hears the broadcast and wakes up. Its job: figure out which contracts this signal might matter to.

It pulls down the active correlation rules - about 30 of them in production - and evaluates each one. Rules look like:

> *'If a weather.severe_warning covers an emirate where one of our contracts has a delivery obligation in the next 14 days, AND the contract has a force majeure clause that lists severe weather as an excused event, then THIS signal is correlated to THAT contract.'*
>
> *'If a payment_delay exceeds 21 days AND the contract's payment_terms clause grants the supplier a cure period, then correlate.'*

For our two signals:

- The weather alert correlates to 3 contracts with active deliveries in Fujairah port that week.
- The payment delay correlates to 1 contract - OQOOD-2026-003.

A row gets written to the correlation table for each match. Another PG NOTIFY fires.

The point: *correlation is the system's 'this signal might affect these specific contracts' step. Nothing else happens without it.*

Stage 3 - CLASSIFY "How bad is it?"

Two workers wake up on this notify.

The risk-case-auto-create worker checks: is this correlation severe enough to need a human looking at it? It uses a 4-tier classifier (Tier 1 = monitor passively -> Tier 4 = executive must act today). The weather correlation hits Tier 2. The payment delay hits Tier 3. Both spawn a risk case row, auto-assigned to the contract's drafter - Hala.

The score-recompute worker rebuilds the 5-dimensional risk score for each affected contract. This is the bit worth understanding properly.

The 5-dimensional risk score (how it's built)

Each contract carries five separate scores from 0 to 100. Each score is calculated as probability x impact, summed across whichever rules contributed.

Dimension	What it measures	Which rules feed it
Legal	*"Could we get sued or breach a regulation here?"*	Sanctions + regulatory rules
Financial	*"Are we losing money or about to?"*	Market disruption + counterparty defaults
Operational	*"Can we still deliver on time?"*	Geopolitical + cyber + supply chain
Reputational	*"Will this make us look bad publicly?"*	Geopolitical + counterparty news
Compliance	*"Are we drifting out of policy?"*	Sanctions + regulatory + cyber

These aren't five completely separate signals - one rule can contribute to multiple dimensions. The weather alert mainly bumps Operational (delivery risk) and Reputational (force majeure invocation is visible). The payment delay bumps Financial sharply, plus a smaller nudge to Operational.

The five dimension scores roll up into:

- **Composite score** - a weighted average. Weights are tunable from the Platform Admin 'Scoring weights' page.
- **MaR - Money at Risk** - composite x contract value. The dirham number that turns a score into a board-ready figure.
- **AVaR - Aggregate Value at Risk** - sum of MaR across the whole portfolio, for the Executive dashboard.

All of this lands in a materialized view (latest_risk_score) so the dashboards stay fast even as scores update constantly.

Stage 4 - EXPLAIN "Tell me what to do"

Now the AI layer runs. When Hala opens the contract, she sees a new 'Critical Impact' frame at the top. The Advisory Drafter has already pre-composed a draft message - for example:

> *'OQOOD-2026-003 is exposed to a weather force majeure event between 12 Jun and 15 Jun. Clause Section 12.1 lists severe weather as an excused event with a 5-business-day notice requirement. Recommend issuing a Force Majeure Notice to Mubadala by close of business 12 Jun.'*

That paragraph is half template (a Mustache template stored in advisory_template) and half LLM-generated (gpt-4o). Before the LLM runs, two safety layers fire:

- **ACL pre-filter** - only contracts Hala is allowed to see are passed into the prompt. Other people's contracts are blocked at the SQL layer, not just the UI.
- **Scope-hash audit** - a hash of 'what the AI was allowed to see' is logged. If anyone later asks 'did the AI see contract X?', the audit answers it.

The AI Risk Assistant sits in the bottom-right corner. Hala can ask it 'show me everything force majeure related across my book' and it answers using pgvector to find clauses that semantically match - not just keyword search. Every citation it shows links back to the actual clause.

Stage 5 - NOTIFY "Get someone's attention"

Finally, the notification dispatcher fans out:

- **In-app** - badge for Hala - she sees a '2 new' indicator on the bell.
- **Email** - to her via SMTP (Nodemailer).
- **Teams + Slack** - payloads are composed in real Adaptive Card / Block Kit JSON and captured to notification_dispatch_log. In production these go to actual channels; in the demo they sit in the log so you can prove the wiring works without spamming anyone's chat.
- **Executive routing** - for Eman, only the Tier 3+ payment delay notification is dispatched - she's subscribed to high-priority only. The weather alert stays at Hala's level.

If an email bounces, the notification-retry worker re-tries on a 1 min -> 5 min -> 30 min -> 2 hr -> 8 hr backoff before giving up. Five attempts spread over ten and a half hours.

Putting it together

By 08:00 AM, Hala opens her laptop and her dashboard already shows:

- 2 new risk cases assigned to her
- OQOOD-2026-003 flagged as High risk - Financial dimension jumped from 38 -> 71
- Two pre-drafted advisory notices waiting for her to review and approve
- A Teams card in the captured log demonstrating what would have been sent to the Mubadala account team

She didn't read a single news article. She didn't look at SAP. She didn't manually score anything. The platform did the boring work between 07:42 and 07:58.

One-sentence version

Public feeds and the customer's own systems both drop signals into one inbox; rules correlate those signals to specific contracts; risk gets scored across five dimensions and surfaced as a risk case; AI drafts the advisory; notifications fan out - and every step is audited and bounded by who's allowed to see what.

PART 2

Explaining the architecture

Three concepts on the Logical Architecture slide that don't explain themselves

---

Slide 2 uses two software-engineering labels - 'Cross-cutting' and 'Async backbone' - that mean little to people outside the field. Customers also reasonably ask why this is a single deployable instead of a microservices architecture. These notes give plain-language versions you can deliver directly, with the framing you can repeat as the customer asks follow-ups.

1. "Cross-cutting" - what it actually means

Better label: *Always-on safeguards* or *Per-request checks that run on every action*.

Think of an airport. Every passenger - first class or economy, business or leisure, domestic or international - goes through the same security, the same passport check, the same boarding-pass scan. Nobody skips those. They're not features, they're the rules of the building.

That's what these are. Every API call into the platform hits the same checkpoint stack before any actual work begins, and every action gets recorded on the way out.

Five gates on the way IN:

- **Are you who you say you are?** - JWT auth
- **Are you allowed to do this specific thing?** - RBAC permission codes
- **Which tenant's data are you allowed to see?** - RLS GUC - enforced by Postgres, not the app
- **Did you send a valid request body?** - Zod validation
- **Are you hammering us too fast?** - Rate limiting

Five things recorded on the way OUT:

- **Who did what, when, with before/after values** - audit_log fan-out
- **Every log line tagged with the request ID** - request correlation
- **Sensitive fields stripped before logging** - Pino redaction
- **Traces shipped to your observability stack** - OpenTelemetry
- **UI labels and DB messages translated** - i18n EN + AR

The point: *these are the ten things that run on every single API call automatically. They're not features - they're the platform's rules of the road. You can't bypass them, you can't forget them, and we didn't have to write them ten times for ten different features.*

Why this matters to the customer's tech team:

If they ask 'how do you stop developer mistakes from leaking data across tenants?' - the answer is this. RLS at the database layer means even a buggy controller cannot return another tenant's rows.

2. "Async backbone" - what it actually means

Better label: *Background workers* or *Behind-the-scenes engine*.

Restaurant analogy. When you order food, the waiter writes it down and says 'got it' within seconds. But the kitchen is cooking in the background for the next twenty minutes. You sit at your table doing other things. When the food is ready, the waiter brings it over.

Same idea here. Some work the user MUST wait for - 'save my contract', 'show me the dashboard', 'approve this request'. Those need to finish in under a second. But some work takes 30 seconds, or 5 minutes, or 8 hours. Examples:

- Extracting text from a 50-page PDF and running OCR on it
- Calling OpenAI to embed 500 clauses for semantic search
- Re-running risk scores when a new sanctions update lands
- Retrying a failed email send tomorrow morning if SMTP was down
- Polling 14 external news feeds every few minutes

If the user had to sit watching a spinner for any of those, the product would feel broken. So the platform's design is:

- User clicks a button
- The API responds in 200 milliseconds: 'got it, we're on it'
- A background worker picks the work up
- When the work is done, the user gets a notification - or sees the result the next time they open the relevant page

There are 17 of these workers running. Some are triggered by a clock (every 5 minutes, every hour, every day). Some are triggered by an event - when a new signal lands in the database, the database itself tells the workers 'something new arrived' and they wake up.

The point: *these are the parts of the platform that work for you while you're doing something else. You don't wait on them, they don't slow your screen down, but they're the reason the platform feels alive instead of reactive.*

Why this matters to the customer's tech team:

When they ask 'how does the platform scale?' - this is the answer. The web tier handles fast user requests; the background tier handles slow batch work. They scale independently, and you only pay for the capacity you need.

3. "Why no microservices?" - the version to deliver

Open with a one-sentence reassurance. Then make three points. Close with a single line that frames it as a deliberate choice.

Setup line

"This is a really fair question, and let me answer it directly. The platform is a single deployable today - but that's a deliberate choice, not a shortcut. Microservices solve a specific kind of problem, and that problem isn't the one we have right now."

Point 1 - Microservices solve an organisational problem, not a technical one.

"Microservices were invented at Amazon and Netflix because they had hundreds of teams that each needed to ship on their own schedule without stepping on each other. Splitting a system into 50 services let each team own one, deploy whenever they wanted, and use whatever language they wanted. That's the real benefit - independent team velocity. The technical benefits you read about - scaling, resilience - you can get those without microservices."

Point 2 - The boundary that matters in this platform is the database, not the API.

"Look at the slide. The thick red box is the database - that's where all the business logic lives, around 700 PostgreSQL functions. The Express API above it is intentionally thin: a controller's only job is to call one of those functions. If we split that thin layer into microservices, we'd just be slicing an already-thin layer into thinner slices, while the actual business logic stays in one Postgres database anyway. We'd add network hops, distributed transactions, debugging complexity - and we'd gain nothing. The natural boundary, the place where logic actually lives, is the database. We respected that."

Point 3 - We've already designed for the day you actually need to scale.

"What microservices give you for free - independent scaling of different parts - we already have. The web tier and the 17 background workers run in the same Node process today because it's simpler operationally for a single-tenant pilot. But the moment you need to scale them separately - say your OSINT ingestion gets heavy, or you want the API to scale up for a busy hour without spinning up extra worker capacity - that's a configuration change, not a rewrite. Same for read replicas on Postgres, same for swapping the in-memory rate limiter to Redis. Weeks of plumbing, not months of redesign."

Closing line

"Microservices are a tool. We didn't reject them on principle - we just didn't need them yet, and adopting them prematurely would have made the system harder to operate without making it better. If a future module needs its own deployment cadence - say a separate team takes over the risk-intelligence layer - we can carve it out cleanly at that point, because the database is where the contracts between modules live. Until that's a real organisational need, we keep the operational footprint small."

The anchor sentence

If the customer remembers nothing else from the architecture conversation, this is the line worth leaving them with. Memorise it.

> ***The platform is built around one core idea: business logic belongs in the database, and everything else - the API, the workers, the cross-cutting checks, the integrations - is a thin layer arranged around it. That single decision is what makes the architecture simple to operate, hard to misuse, and easy to scale.***

That one sentence covers all three explanations - cross-cutting, async backbone, and the microservices question. It's the through-line of the whole architecture.